

# Computer Science & IT

## Database Management System



**Transactions & concurrency control**

**Lecture No. 06**

**By- Vishal Sir**





# Recap of Previous Lecture



Topic

Classification based on recoverability

Topic

Classification based on serializability



Topic

Conflicting and non-conflicting operations

Topic

Conflict serializable schedule

Topic

Precedence graph



# Topics to be Covered



- ✓ **Topic** Precedence graph
- ✓ **Topic** Topological order
- ✓ **Topic** Testing condition for Conflict serializability
- ✓ **Topic** Practice question







## Topic : Classification of schedule

### Classification based on Recoverability

- ① Irrecoverable Schedule
- ② Recoverable Schedule
- ③ Cascadeless Rollback Recoverable Schedule
- ④ Strict Recoverable Schedule

Already done

### Classification based on Serializability

- ① Conflict Serializable Schedule
- ② View Serializable Schedule

Classification based on Serializability

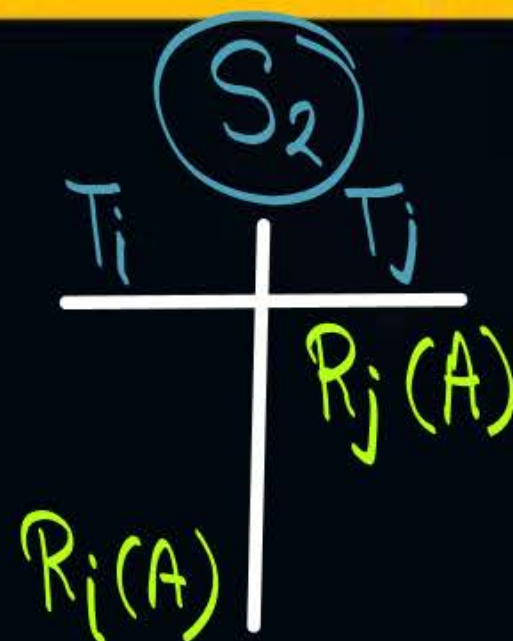
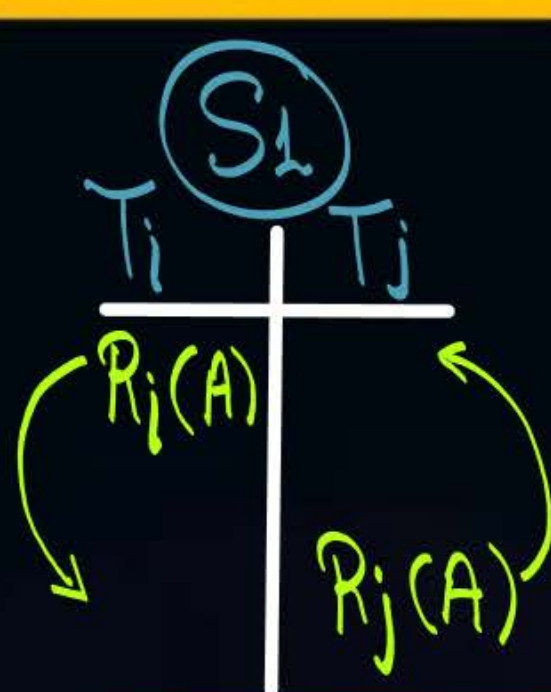
Conflict Serializable Schedules  
(C.S.S.)





# Topic : Conflicting and non-conflicting operations

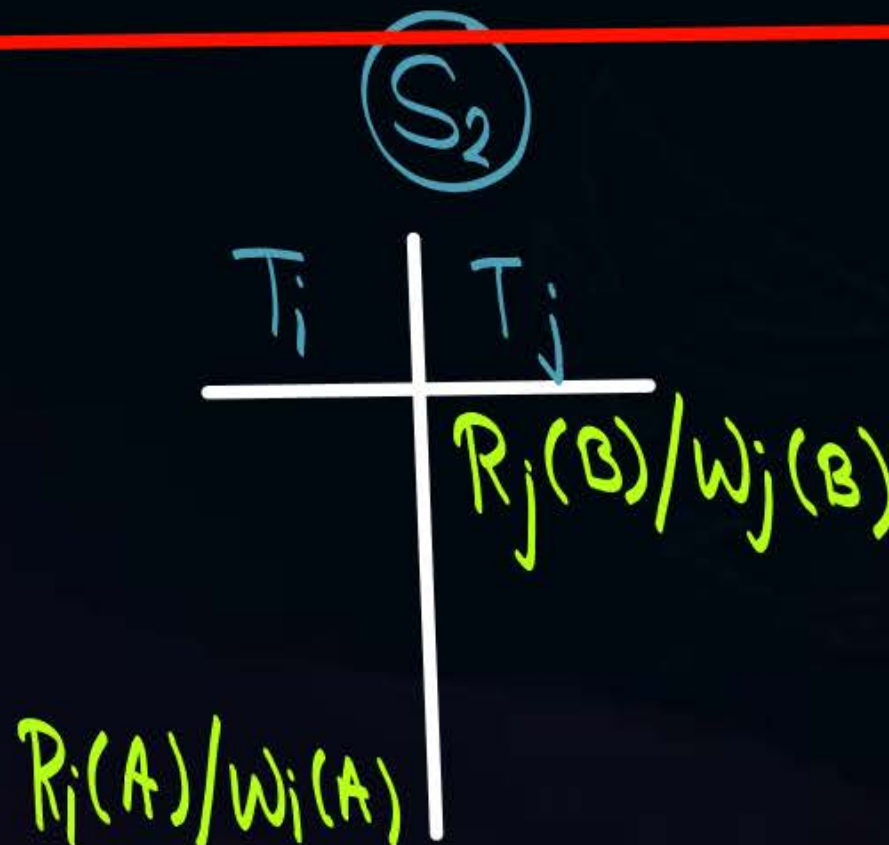
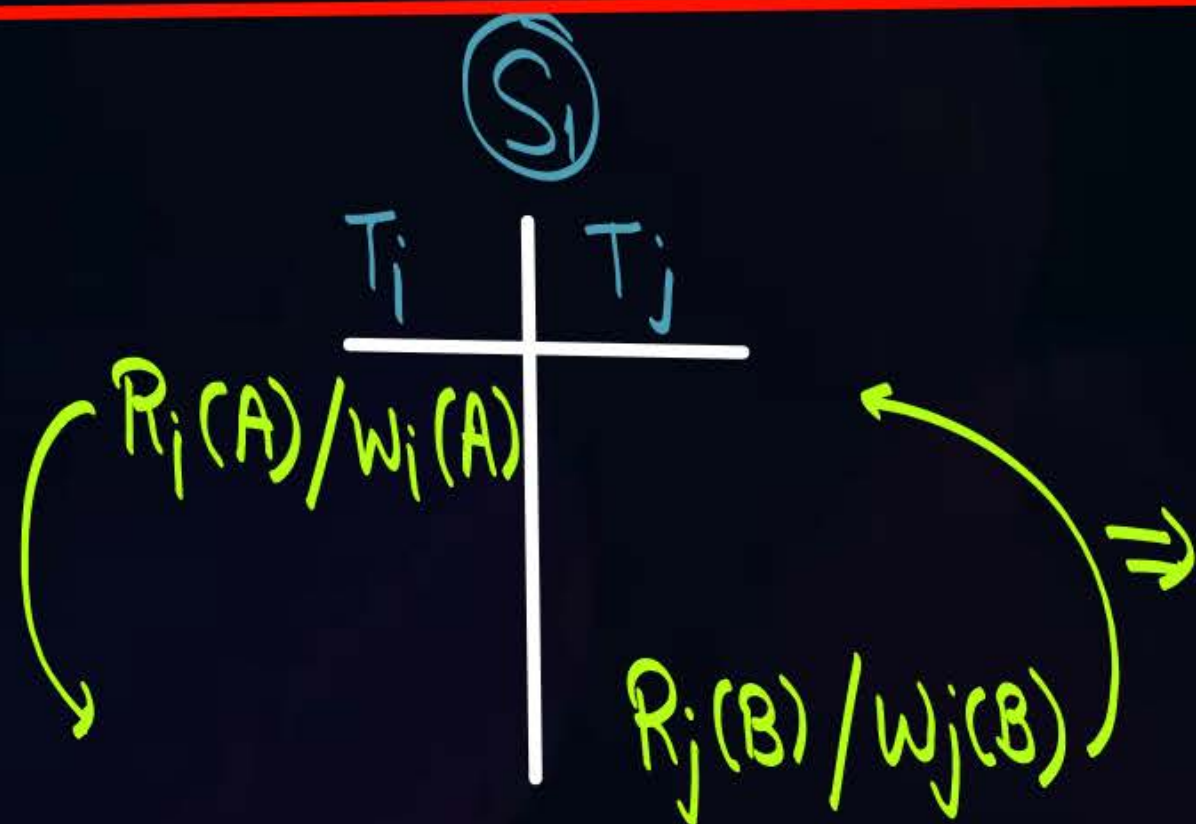
①



$$S_1 \equiv S_2$$

∴  $R_i(A)$  &  $R_j(A)$  are not Conflicting op<sup>n</sup>

②



$$S_1 \equiv S_2$$

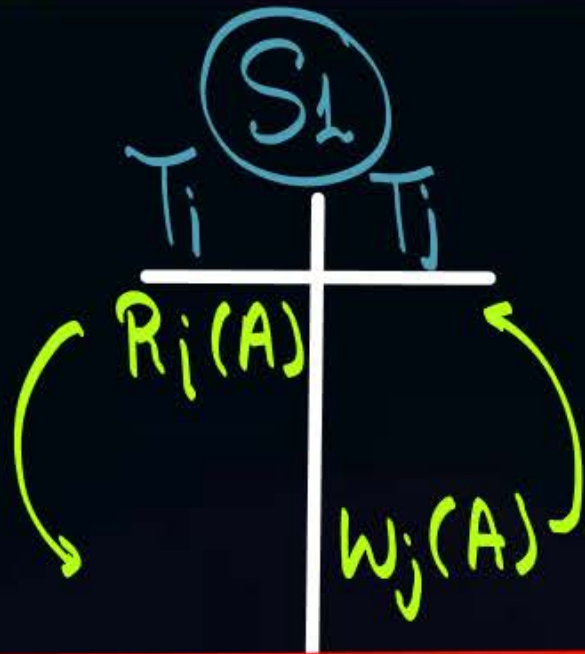
∴  $R_i(A)/W_i(A)$  is non-conflicting with  $R_j(B)/W_j(B)$



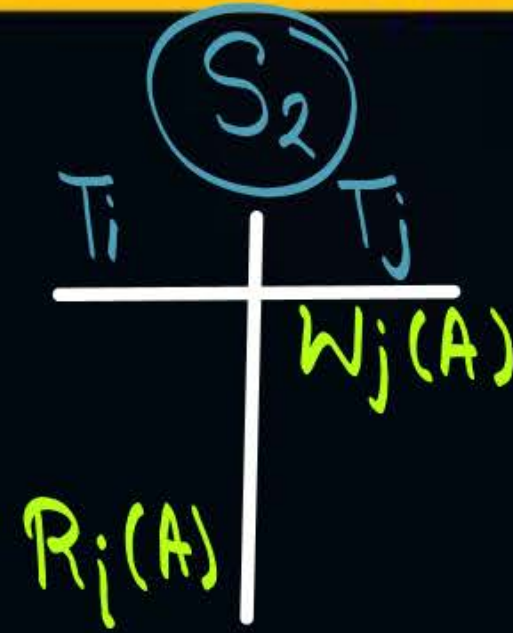


## Topic : Conflicting and non-conflicting operations

③

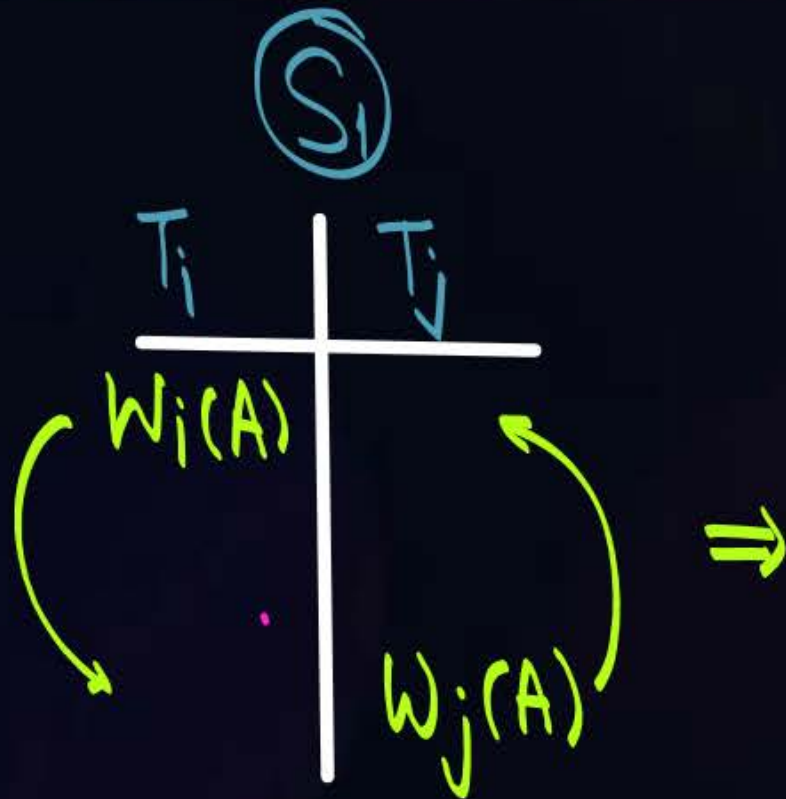


⇒



$S_1 \neq S_2$  ∴ R<sub>i</sub>(A) & W<sub>j</sub>(A) are conflicting op.

④



⇒



$S_1 \neq S_2$  ∴ W<sub>i</sub>(A) & W<sub>j</sub>(A) are conflicting op.





## Topic : Conflicting and non-conflicting operations

✓ **Conflicting Operations** : Two operations are called as conflicting operations if all the following conditions hold true for them -

- ✓ ☐ Both the operations belong to different transactions
- ✓ ☐ Both the operations are on the same data item
- ✓ ☐ At least one of the two operations is a write operation



## Topic : Conflicting and non-conflicting operations

- **Non-Conflicting Operations** : Two operations are non-conflicting operations only if
- ✓ ☒ Both the operations are on different data items
  - or
  - ✓ ☒ Both the operations are read operations



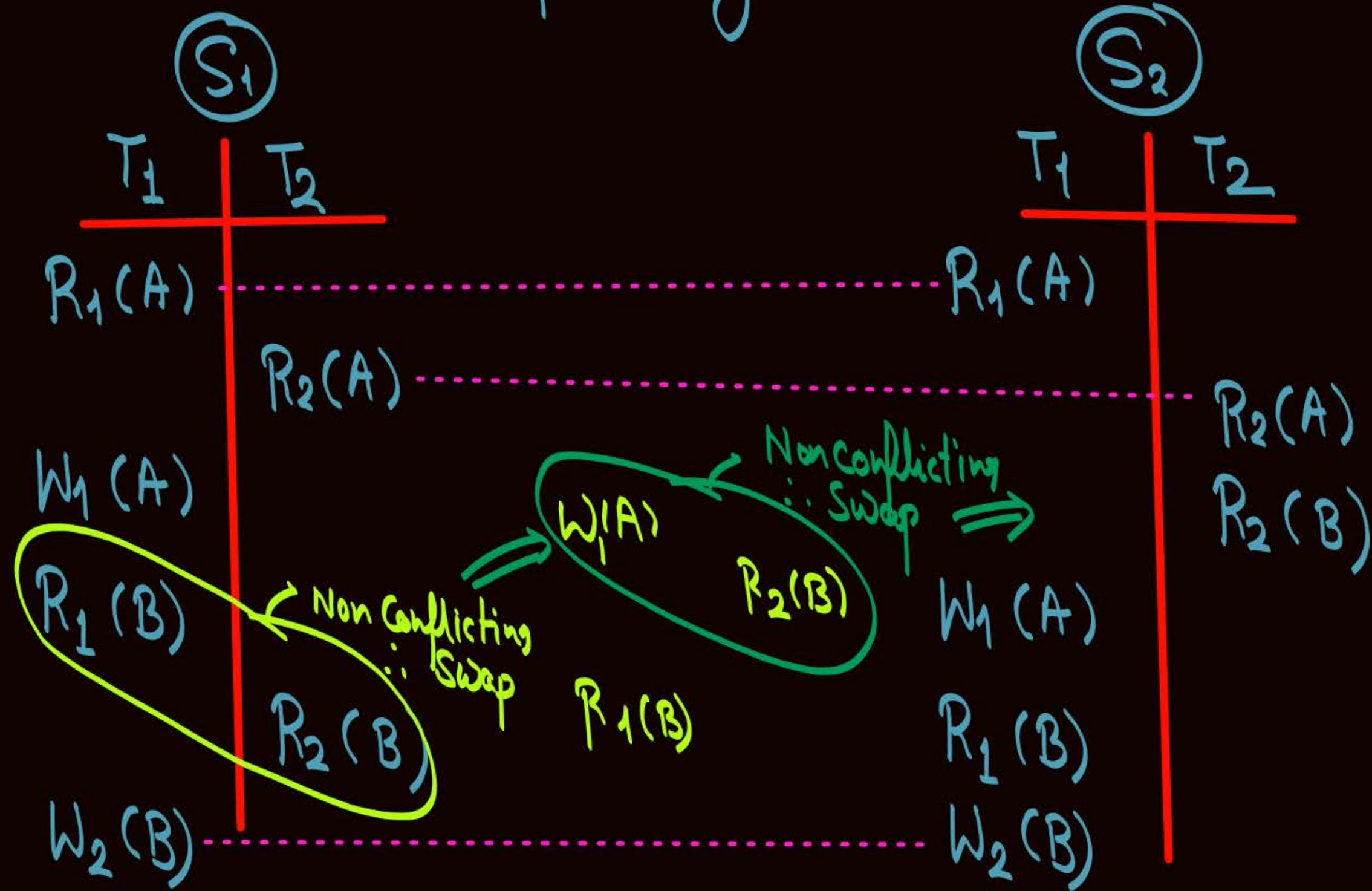


## Topic : Conflict equivalent schedule

Consider two schedules  $S_i$  and  $S_j$ ,

If one can be transformed into another by swapping the position of some of the non-conflicting operations in it, then schedule  $S_i$  and  $S_j$  are called conflict equivalent schedule.

eg Consider the following Schedules



- $S_1$  can be converted into  $S_2$  by swapping the position of some of the non-conflicting operations.
- ∴  $S_1$  is Conflict Equivalent to  $S_2$



eg Consider the following Schedules

(S<sub>1</sub>)

| T <sub>1</sub>     | T <sub>2</sub>     |
|--------------------|--------------------|
| R <sub>1</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |
|                    | R <sub>2</sub> (A) |
| R <sub>1</sub> (B) |                    |
| W <sub>2</sub> (B) |                    |
|                    | R <sub>2</sub> (B) |

Conflicting  
Pair of opn  
∴ we can  
not swap

⇒ Hence  
S<sub>1</sub> can not be transformed into S<sub>2</sub>

(S<sub>2</sub>)

| T <sub>1</sub>     | T <sub>2</sub>     |
|--------------------|--------------------|
| R <sub>1</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |
|                    | R <sub>2</sub> (A) |
|                    | R <sub>2</sub> (B) |
| R <sub>1</sub> (B) |                    |
| W <sub>2</sub> (B) |                    |

for Conflictint Equivalence  
Swapping of Conflicting  
opn is not allowed.  
∴ S<sub>1</sub> & S<sub>2</sub> are  
not Conflict Equivalent  
to each other





## Topic : Conflict serializable schedule

- If a given schedule can be converted into *any* serial schedule by swapping the positions of some of its non-conflicting operations, then it is called a conflict serializable schedule.

i.e. If given schedule is conflict equivalent to *any* of the serial schedule, then schedule is called Conflict serializable schedule.



eg Consider the following Schedules

(S<sub>1</sub>)

| T <sub>1</sub>     | T <sub>2</sub>     |
|--------------------|--------------------|
| R <sub>1</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |
|                    | R <sub>2</sub> (A) |
| R <sub>1</sub> (B) |                    |
| W <sub>2</sub> (B) |                    |
|                    | R <sub>2</sub> (B) |

Conflicting op<sup>n</sup>

∴ R<sub>2</sub>(A) Can not be moved above W<sub>1</sub>(A),  
Hence S<sub>1</sub> Can not be converted into  
Serial Schedule T<sub>2</sub> → T<sub>1</sub>

R<sub>2</sub>(A) is non-conflicting with  
R<sub>1</sub>(B) as well as W<sub>1</sub>(B),  
∴ we can swap the position  
and we can move R<sub>2</sub>(A) below  
the op<sup>n</sup> R<sub>1</sub>(B) & W<sub>1</sub>(B),  
and resulting schedule will be  
Serial Schedule T<sub>1</sub> → T<sub>2</sub>

Schedule S<sub>1</sub> is conflict equivalent to at least one  
serial schedule {i.e., T<sub>1</sub> → T<sub>2</sub>}, Hence Schedule S<sub>1</sub> is  
a Conflict serializable schedule

T<sub>1</sub> → T<sub>2</sub>

| T <sub>1</sub>     | T <sub>2</sub>     |
|--------------------|--------------------|
| R <sub>1</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |
| R <sub>1</sub> (B) |                    |
| W <sub>1</sub> (B) |                    |
|                    | R <sub>2</sub> (A) |
|                    | R <sub>2</sub> (B) |

T<sub>2</sub> → T<sub>1</sub>

| T <sub>1</sub>     | T <sub>2</sub>     |
|--------------------|--------------------|
|                    | R <sub>2</sub> (A) |
|                    | R <sub>2</sub> (B) |
| R <sub>1</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |
| R <sub>1</sub> (B) |                    |
| W <sub>1</sub> (B) |                    |





## Topic : NOTE



- ① If schedule  $S$  is a Conflict serializable schedule, then it is guaranteed that schedule ' $S$ ' is a Serializable schedule
- ② If schedule  $S$  is not a Conflict serializable schedule, then schedule  $S$  may or may not be a serializable schedule





## Topic : NOTE



- ③ Conflicting operations of transactions will define the precedence order in which transactions must appear in any equivalent serial schedule

eg. . .

| $T_i$    | $T_j$    |
|----------|----------|
| $R_i(A)$ | $W_j(A)$ |

$T_i$  then  $T_j$ , i.e., in any equivalent serial schedule  $T_i$  must execute before  $T_j$



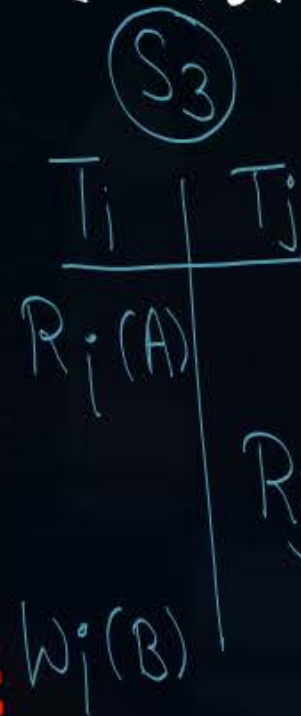
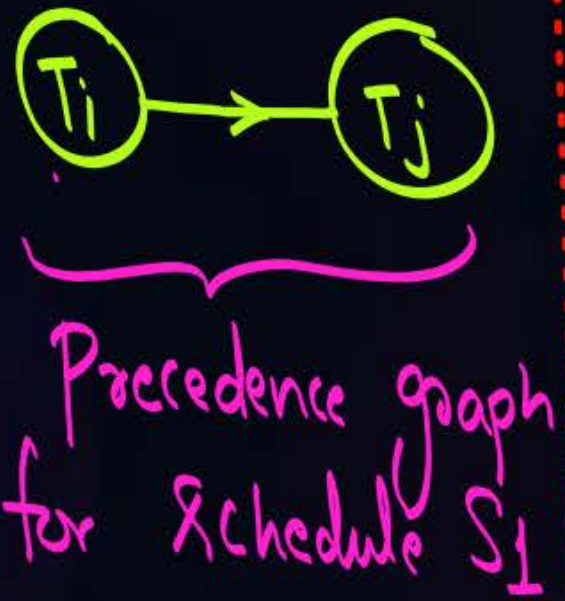


## Topic : Precedence graph

Precedence graph  $G=(V,E)$  is a directed graph.

Where  $V =$  Set of vertices  $=$  Set of transactions of the schedule

$E$ : Set of edges  $\equiv$  Set of directed edges based on the Precedence order defined by Conflicting Operations in the Schedule



- No pair of conflicting op<sup>n</sup> in  $S_3$
- No precedence

Hence No edge

No edge in the precedence graph





## Topic : Testing condition for conflict serializable schedule

- ① \* If precedence graph is acyclic, then schedule will be a Conflict serializable schedule, and hence it will be a serializable schedule
- ② If precedence graph is cyclic, then schedule is not a Conflict serializable schedule, but we can not comment on the serializability of the schedule it may or may not be serializable

Note:

If precedence graph is acyclic, then schedule is a Conflict serializable schedule, and Conflict Equivalent serial schedule's can be given by topological order of the precedence graph.





## Topic : Topological order

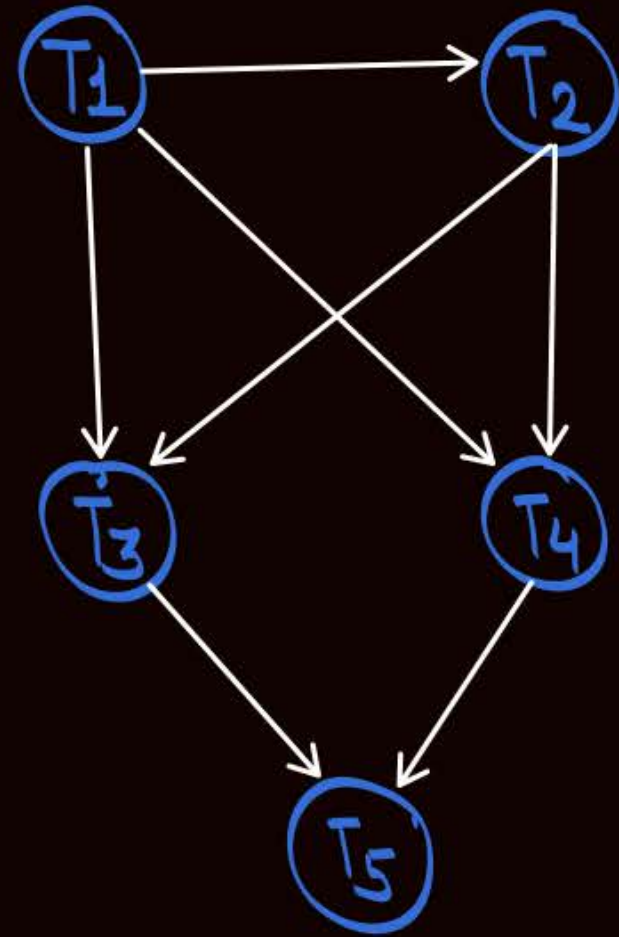
Let  $G = (V, E)$  be an acyclic precedence graph.

Step 1:- Visit a vertex  $V \in G$  such that in-degree of vertex  $V = 0$ , and after visiting that vertex delete it from graph  $G$ .

Step 2:- Repeat step-1 until graph becomes empty.

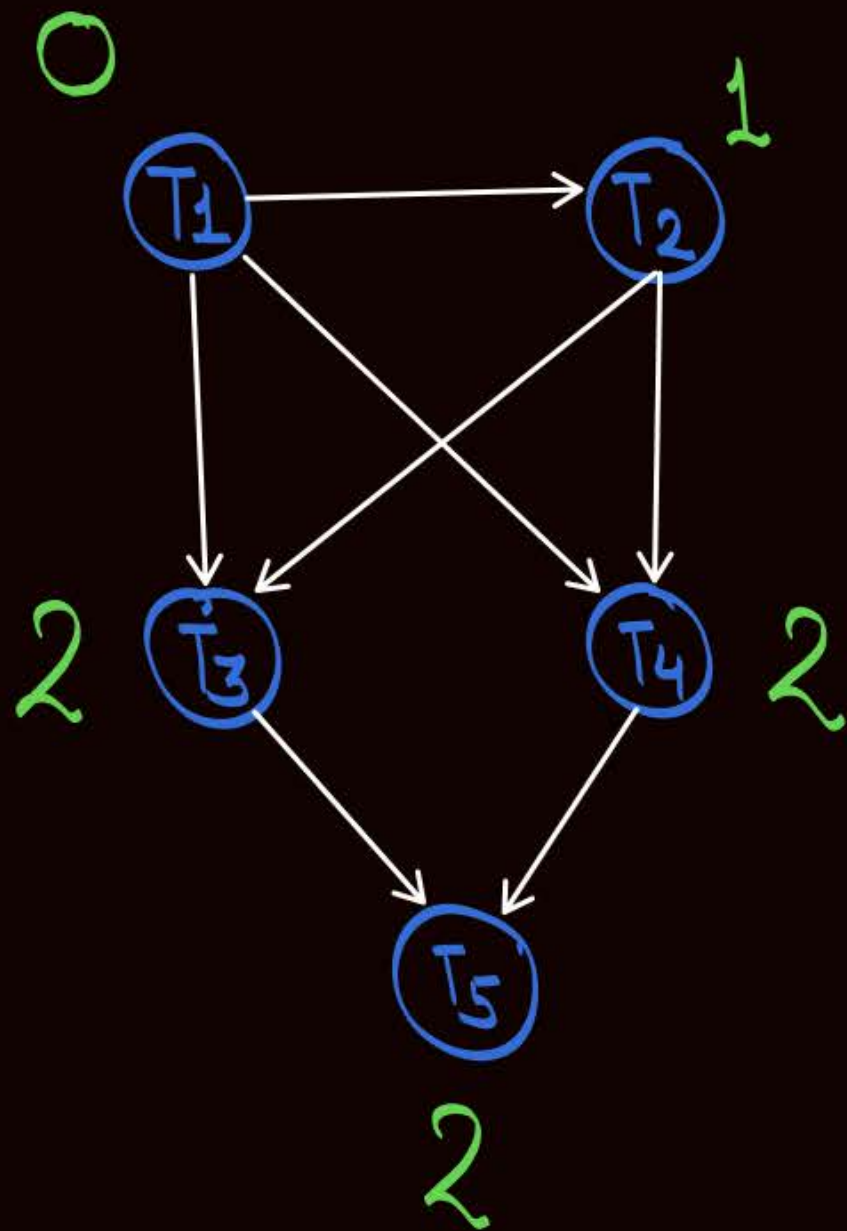
- The order in which vertices of graph  $G = (V, E)$  can be visited will define the topological orders of the given precedence graph.
- There may be more than one topological orders for a given Precedence graph.

→ Consider the following precedence graph w.r.t. schedule 'S'.



Identify all the serial schedules that are conflict equivalent to schedule 'S'.

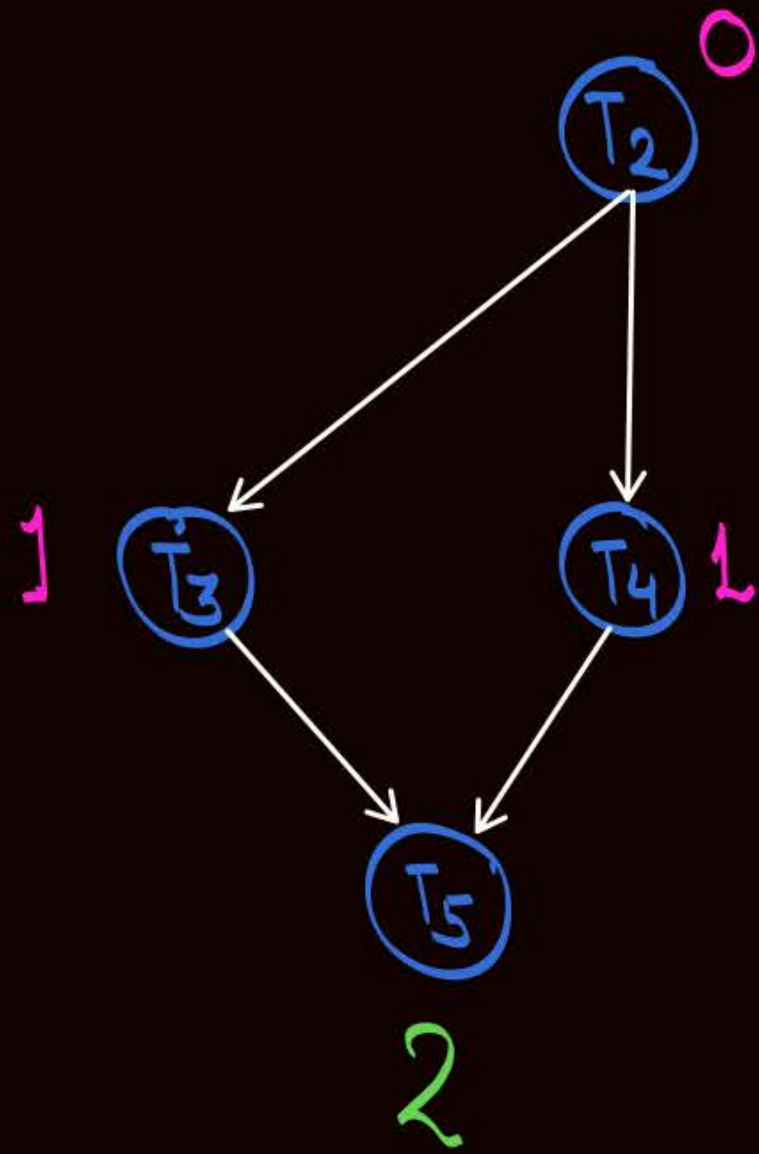




visit



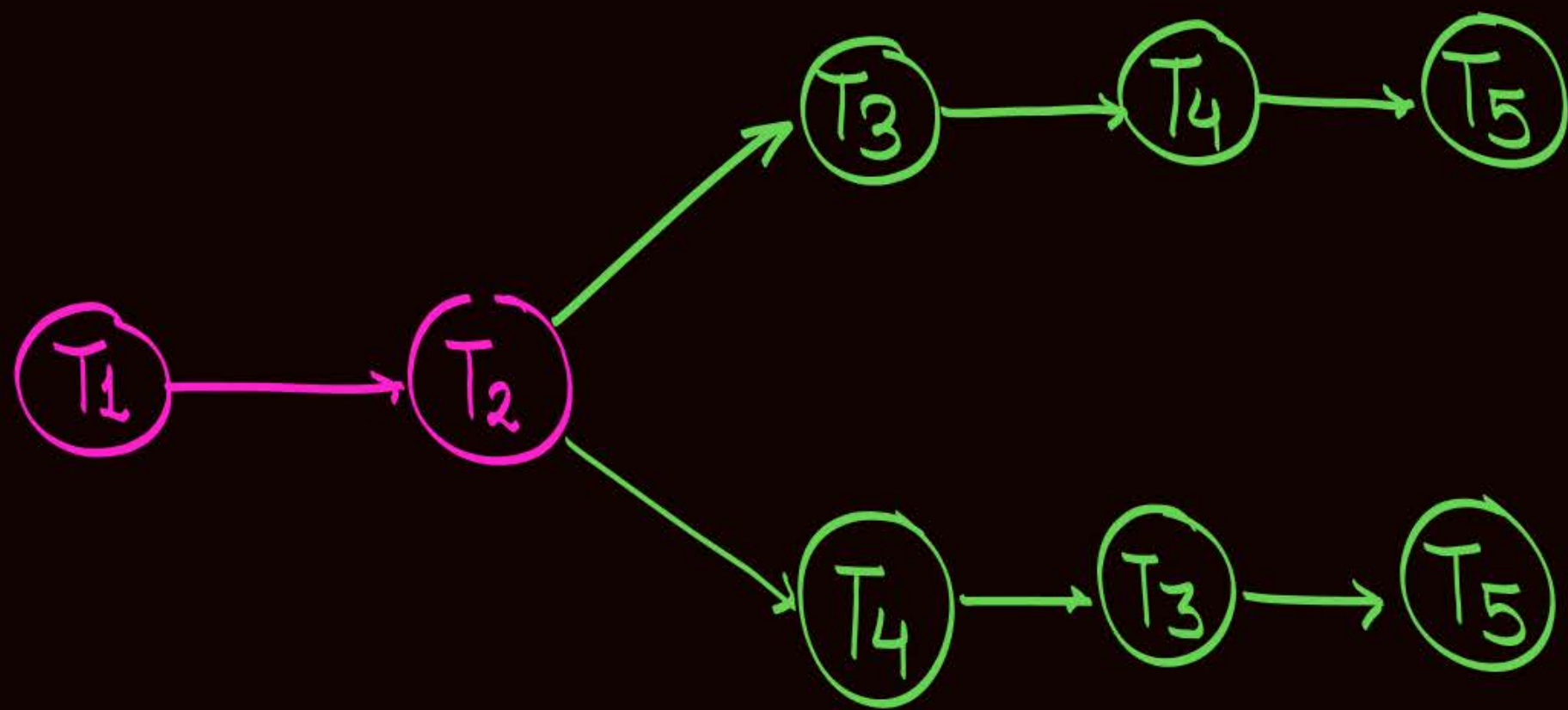
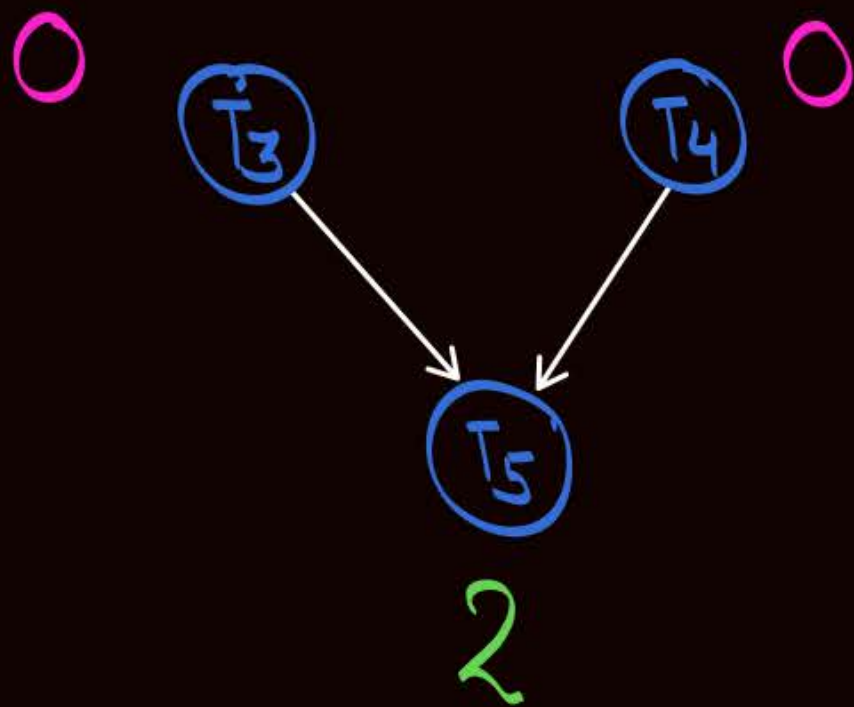
and  
delete it



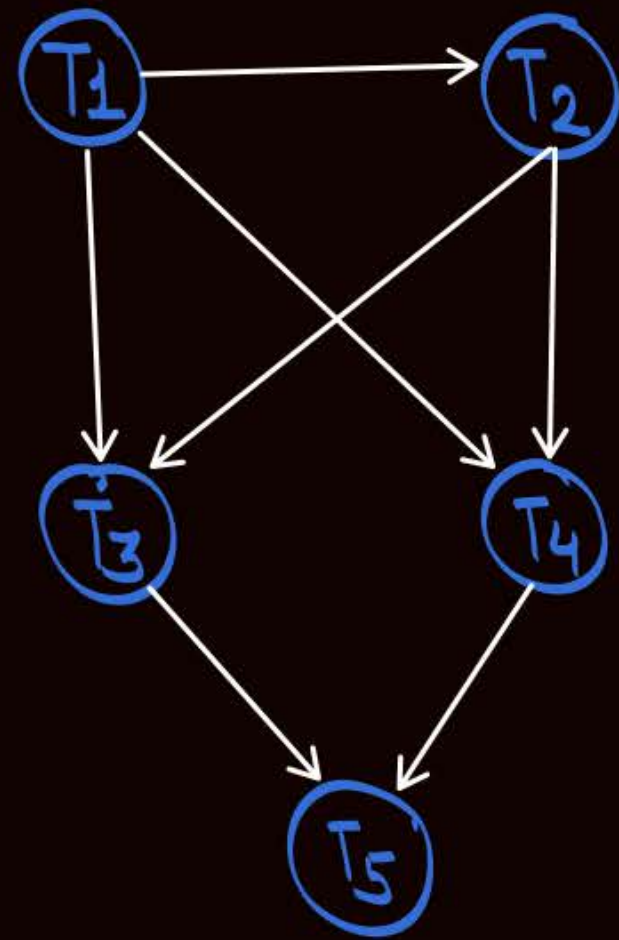
next vertex with  
indegree = 0 is  $T_2$

So, visit it  
and delete it





• Consider the following precedence graph w.r.t. schedule 'S'.



Topological orders are

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$

and

$T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_5$

Two topological orders,  $\therefore$  schedule S is Conflict Equivalent to two serial schedules, and those serial schedules are

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$

$T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_5$

$\left. \begin{array}{l} \text{ic.} \\ \text{same as} \\ \text{orders of} \\ \text{precedence} \end{array} \right\} \text{topological orders of the graph}$

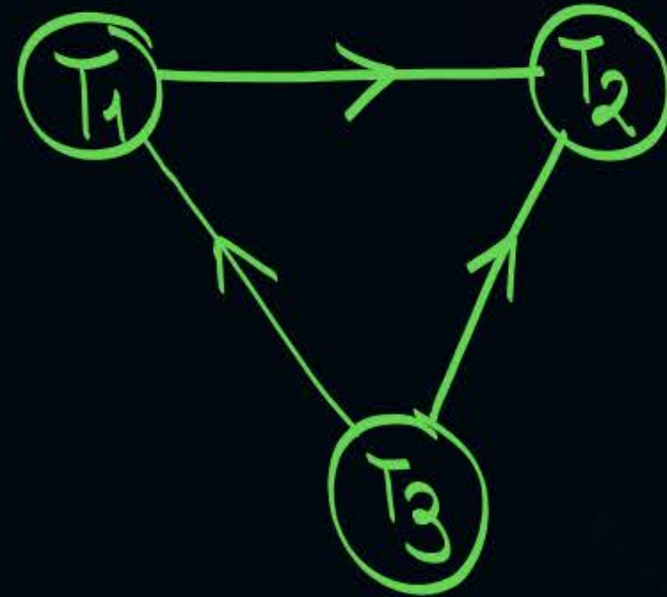
Identify all the serial schedules that are conflict equivalent to schedule 'S'



#Q. Check whether the schedule is conflict serializable schedule or not?

If conflict serializable schedule then identify all conflict equivalent serial schedules.

| $T_1$                | $T_2$                                        | $T_3$                |
|----------------------|----------------------------------------------|----------------------|
| $R_1(x)$<br>$W_1(x)$ |                                              | $R_3(y)$<br>$R_3(z)$ |
| $R_1(y)$<br>$W_1(y)$ | $R_2(z)$                                     | $W_3(y)$<br>$W_3(z)$ |
|                      | $R_2(y)$<br>$W_2(y)$<br>$R_2(x)$<br>$W_2(x)$ |                      |



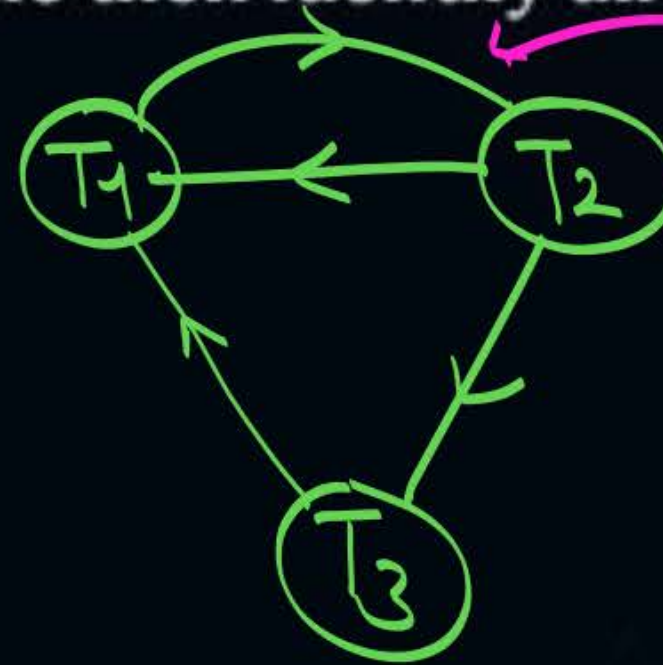
It is an acyclic  
Precedence graph.  
∴ Schedule is a C.S.S.

And conflict equivalent serial schedule is  $T_3 \rightarrow T_1 \rightarrow T_2$   
it is with topological order

#Q. Check whether the schedule is conflict serializable schedule or not?

If conflict serializable schedule then identify all conflict equivalent serial schedules.

| $T_1$    | $T_2$    | $T_3$    |
|----------|----------|----------|
|          | $R_2(A)$ |          |
| $R_1(B)$ |          |          |
|          | $W_2(A)$ |          |
|          |          | $R_3(A)$ |
| $W_1(B)$ |          |          |
| $W_1(A)$ |          |          |
|          | $R_2(B)$ |          |
|          | $W_2(B)$ |          |



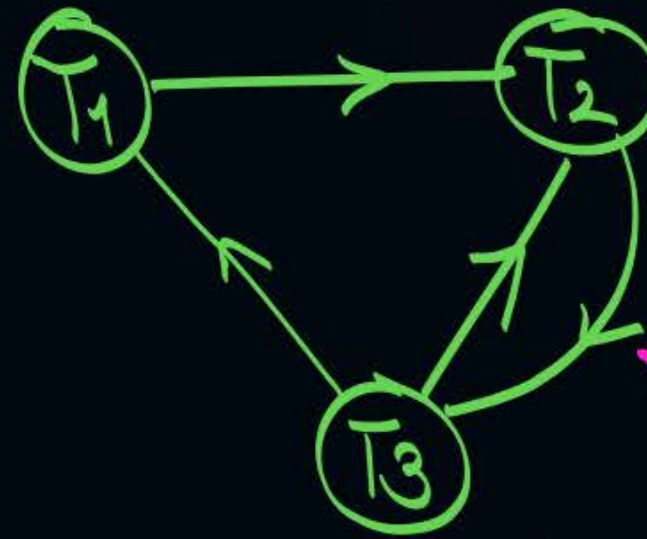
cyclic precedence graph  
is Not a C.S.S.



#Q. Check whether the schedule is conflict serializable schedule or not?

If conflict serializable schedule then identify all conflict equivalent serial schedules.

| $T_1$    | $T_2$    | $T_3$    |
|----------|----------|----------|
|          |          | $W_3(B)$ |
| $R_1(A)$ |          |          |
| $W_1(B)$ |          |          |
|          | $R_2(B)$ |          |
|          | $W_2(C)$ |          |
|          |          | $R_3(C)$ |

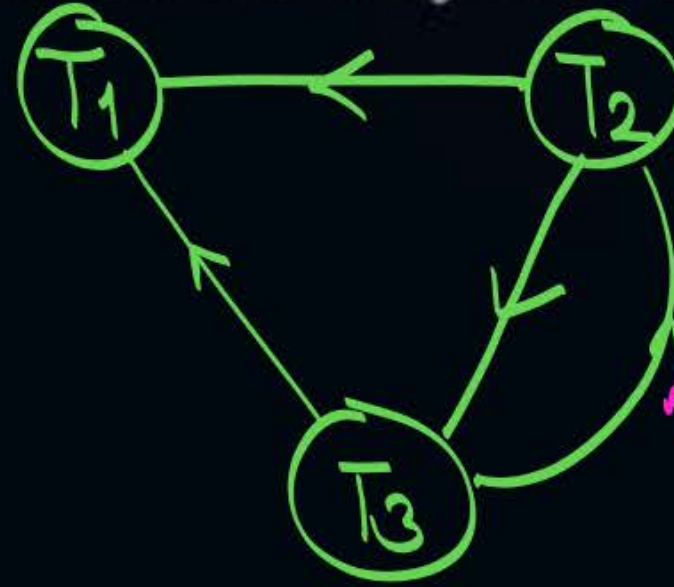


Cyclic precedence graph  
∴ Not a CSS.

#Q. Check whether the schedule is conflict serializable schedule or not?

If conflict serializable schedule then identify all conflict equivalent serial schedules.

| $T_1$    | $T_2$    | $T_3$    |
|----------|----------|----------|
|          | $R_2(z)$ |          |
|          | $R_2(y)$ |          |
|          | $W_2(y)$ |          |
|          |          | $R_3(y)$ |
|          |          | $R_3(z)$ |
| $R_1(x)$ |          |          |
| $W_1(x)$ |          |          |
|          |          | $W_3(y)$ |
|          |          | $W_3(z)$ |
|          | $R_2(z)$ |          |
| $R_1(y)$ |          |          |
| $W_1(y)$ |          |          |



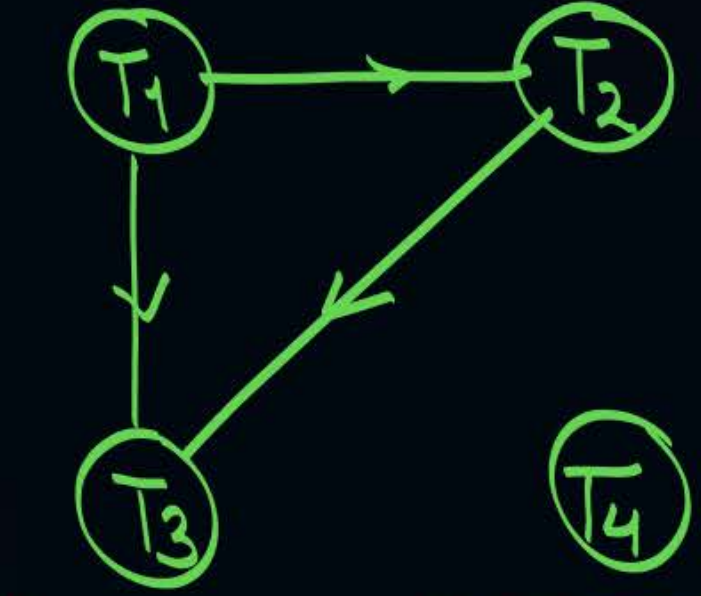
Cyclic Precedence graph  
is Not a C.S.S.



#Q. Check whether the schedule is conflict serializable schedule or not?

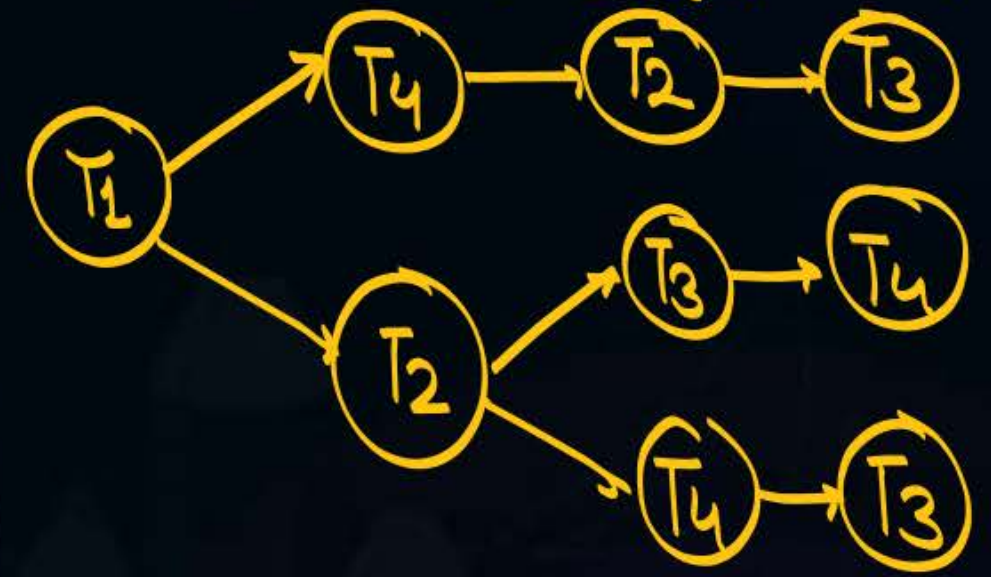
If conflict serializable schedule then identify all conflict equivalent serial schedules.

| $T_1$    | $T_2$    | $T_3$    | $T_4$    |
|----------|----------|----------|----------|
| $R_1(A)$ |          |          |          |
|          | $R_2(A)$ |          |          |
|          |          | $R_3(A)$ |          |
|          |          |          | $R_4(A)$ |
| $W_1(B)$ |          |          |          |
|          | $W_2(B)$ |          |          |
|          |          | $W_3(B)$ |          |



Acyclic precedence graph  
 ∴ Schedule is C.S.S.

Conflict equivalent  
 Serial schedules are



$T_1 \rightarrow T_4 \rightarrow T_2 \rightarrow T_3$   
 $T_4 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3$   
 $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4, T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3$



#Q. Check whether the schedule is conflict serializable schedule or not?

If not conflict serializable then check where the schedule is serializable schedule or not?

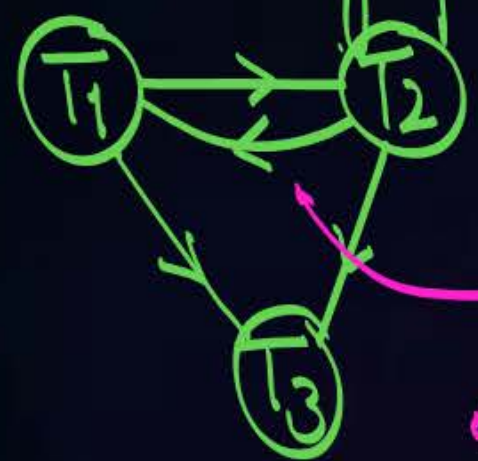
Read from initial DB

| T <sub>1</sub>     | T <sub>2</sub>     | T <sub>3</sub>     |
|--------------------|--------------------|--------------------|
| R <sub>1</sub> (A) |                    |                    |
|                    | W <sub>2</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |                    |
|                    |                    | W <sub>3</sub> (A) |

There are blind write op

A is finally updated by T<sub>3</sub>

Precedence graph



Cyclic  
∴ Not a C.S.S.

Consider the Serial Schedule

T<sub>1</sub> → T<sub>2</sub> → T<sub>3</sub>

| T <sub>1</sub>     | T <sub>2</sub>     | T <sub>3</sub>     |
|--------------------|--------------------|--------------------|
| R <sub>1</sub> (A) |                    |                    |
| W <sub>1</sub> (A) |                    |                    |
|                    | W <sub>2</sub> (A) |                    |
|                    |                    | W <sub>3</sub> (A) |

Read from initial DB

A is finally updated by T<sub>3</sub>

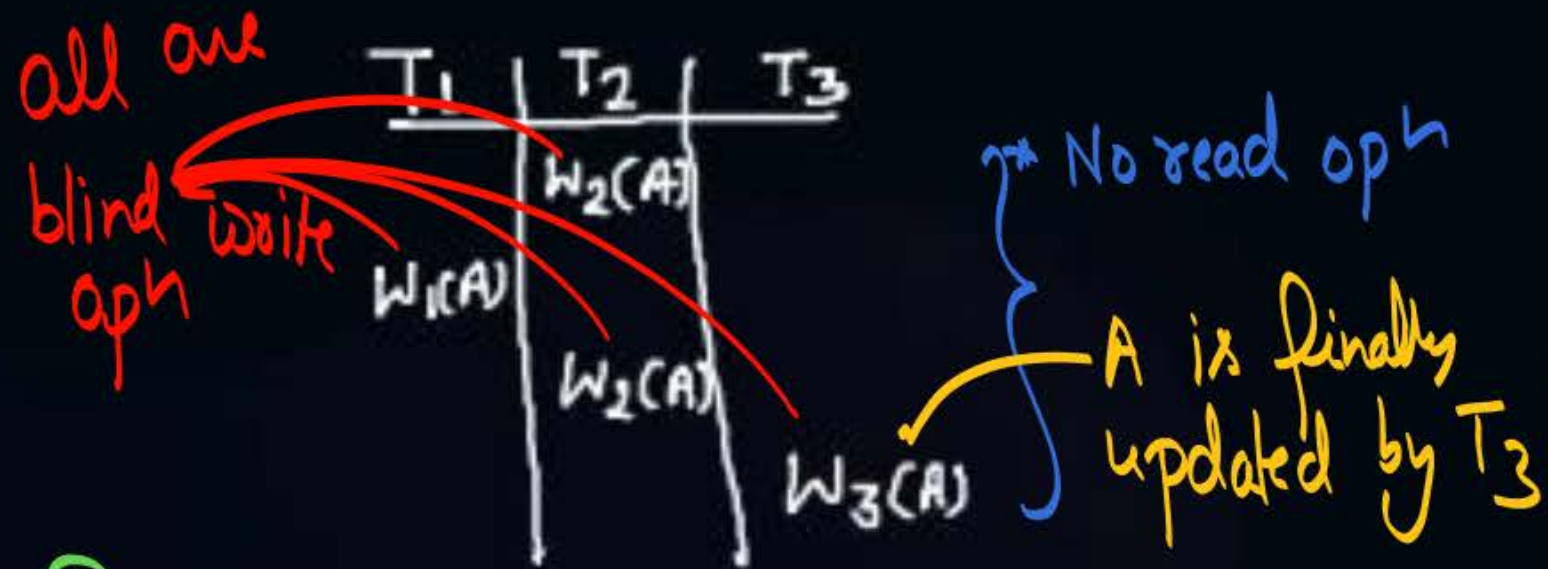
Everything is same as given schedule  
∴ given schedule is equivalent to Serial schedule T<sub>1</sub> → T<sub>2</sub> → T<sub>3</sub>  
Hence, given schedule is a serializable schedule

but → a →

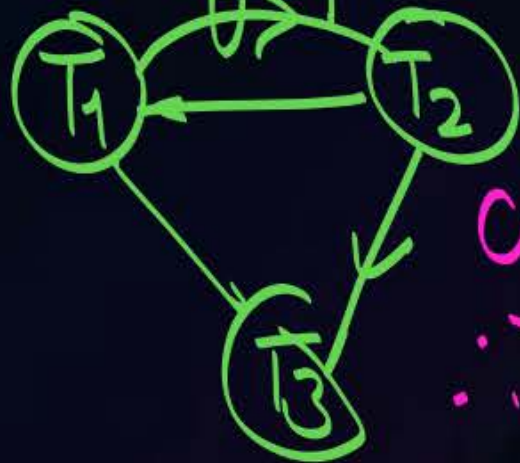


#Q. Check whether the schedule is conflict serializable schedule or not?

If not conflict serializable then check where the schedule is serializable schedule or not?



Precedence graph,



Cyclic  
∴ Not a C.S.S.

but a

$T_1 \rightarrow T_2 \rightarrow T_3$

| T <sub>1</sub>     | T <sub>2</sub>     | T <sub>3</sub>     |
|--------------------|--------------------|--------------------|
| W <sub>1</sub> (A) |                    |                    |
|                    | W <sub>2</sub> (A) |                    |
|                    | W <sub>2</sub> (A) |                    |
|                    |                    | W <sub>3</sub> (A) |

W<sub>1</sub>(A)

W<sub>2</sub>(A)

W<sub>2</sub>(A)

W<sub>3</sub>(A)

$T_2 \rightarrow T_1 \rightarrow T_3$

| T <sub>1</sub>     | T <sub>2</sub>     | T <sub>3</sub>     |
|--------------------|--------------------|--------------------|
|                    | W <sub>2</sub> (A) |                    |
|                    | W <sub>2</sub> (A) |                    |
| W <sub>1</sub> (A) |                    |                    |
|                    |                    | W <sub>3</sub> (A) |

W<sub>2</sub>(A)

W<sub>2</sub>(A)

W<sub>1</sub>(A)

W<sub>3</sub>(A)

Both  $T_1 \rightarrow T_2 \rightarrow T_3$  &  $T_2 \rightarrow T_1 \rightarrow T_3$  are equivalent to given schedule

Hence, Schedule is a Serializable schedule



Note:- ① If schedule is not a Conflict serializable schedule, but a serializable schedule, then at least one blind write op<sup>n</sup> will exist in that schedule

② If a schedule is not a Conflict serializable schedule and a blind write op<sup>n</sup> exists in the schedule, then also schedule may be a non-serializable schedule

③ If schedule is not a C.S.S, and no blind write op<sup>n</sup> exists in the schedule, then schedule can never be a serializable schedule



Note: Conflict serializability Cond<sup>n</sup> is an "if-then" Cond<sup>n</sup>,

i.e, ① If Conflict serializable, then serializable

"if is true"

then

then is also true

"if is false"

then

then may or may not be true

Note: - There are many serializable schedules that are not covered by the definition of Conflict serializability.  
∴ We define "View serializability"

• View serializability Cond<sup>n</sup> is if and only if Cond<sup>n</sup>.

i.e., Schedule is View serializable Schedule if and only if

i.e. View serializability cond<sup>n</sup> will cover all serial Schedules

Schedule is a serializable Schedule



View Serializable Schedule



## 2 mins Summary



- ✓ **Topic** Precedence graph
- ✓ **Topic** Topological order
- ✓ **Topic** Testing condition for Conflict serializability
- ✓ **Topic** Practice question



**THANK - YOU**